

Week 3 Report: Building and Securing a REST API

Team: 3

Introduction to API Security

An API (Application Programming Interface) is a system that allows different applications to communicate and exchange data over a network using HTTP. Because APIs expose data and functionality to external clients, security is a fundamental concern in API design.

The MoMo SMS API handles real mobile money transaction data extracted from 1,691 SMS records. Every endpoint in this API is protected to ensure that only authorized clients can read, create, update, or delete transaction records.

Key Security Threats

Unauthorized Access — Without authentication anyone who knows the API URL can access all transaction data. For a financial system containing real transaction amounts, senders, and receivers this is a serious risk.

Data Tampering — Without protection on write endpoints malicious actors could modify or delete transaction records compromising the integrity of financial data.

Credential Interception — When credentials are transmitted over unencrypted connections attackers can intercept them and gain unauthorized access.

Brute Force Attacks — Without rate limiting attackers can repeatedly try different username and password combinations until they find valid credentials.

API Endpoint Documentation

The MoMo SMS API exposes five endpoints for managing transaction data. All endpoints require Basic Authentication. The base URL is `http://localhost:8080`.

Authentication

Every request must include a valid username and password encoded in base64 in the Authorization header. Requests without valid credentials receive a 401 Unauthorized response.

```
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$ curl -u wronguser:wrongpass http://localhost:8080/transactions
{"error": "Unauthorized. Invalid credentials."}
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$
```

Username: admin

Password: MomoSms26

Endpoints Overview

Method	Endpoint	Description	Success Code
GET	/transactions	Returns all transactions	200
GET	/transactions/{id}	Returns one transaction	200
POST	/transactions	Adds a new transaction	201
PUT	/transactions/{id}	Updates a transaction	200
DELETE	/transactions/{id}	Deletes a transaction	200

```
PROBLEMS  DEBUG CONSOLE  TERMINAL  PORTS
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$ python api/server.py
MoMo SMS API running on http://localhost:8080
Press CTRL+C to stop the server
127.0.0.1 - - [30/May/2026 17:12:42] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:16:49] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:18:15] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:20:36] "GET /transactions HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:21:37] "GET /transactions HTTP/1.1" 401 -
127.0.0.1 - - [30/May/2026 17:28:16] "GET /transactions/1 HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:29:06] "POST /transactions HTTP/1.1" 201 -
127.0.0.1 - - [30/May/2026 17:29:54] "PUT /transactions/1 HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:30:34] "DELETE /transactions/9999 HTTP/1.1" 200 -
```

Error Codes

| Code | Meaning |

|---|---|

| 200 | Request successful |

| 201 | Resource created successfully |

| 400 | Bad request — missing field or duplicate ID |

| 401 | Unauthorized — invalid or missing credentials |

| 404 | Resource or endpoint not found |

GET /transactions

Returns all transactions stored in the system.

Request: curl -u admin:MomoSms26 <http://localhost:8080/transactions>

```
PROBLEMS  DEBUG CONSOLE  TERMINAL  PORTS
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$ curl -u admin:MomoSms26 http://localhost:8080/transactions | head -c 2000
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left   Speed
1[{"transaction_id": 1, "original_transaction_id": "76662021700", "category": "Incoming Money", "amount": 2000, "fee": null,
"balance_after": 2000, "transaction_date": "2024-05-10 16:30:51", "status": "Success", "sender": "Jane Smith", "receiver": "A
bebe Chala CHEBUDIE", "raw_sms": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 2
024-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}, {"transaction
_id": 2, "original_transaction_id": "73214484437", "category": "Payment", "amount": 1000, "fee": 0, "balance_after": 1000, "t
ransaction_date": "2024-05-10 16:31:39", "status": "Success", "sender": "Abebe Chala CHEBUDIE", "receiver": "Jane Smith", "ra
w_sms": "TxId: 73214484437. Your payment of 1,000 RWF to Jane Smith 12845 has been completed at 2024-05-10 16:31:39. Your new
balance: 1,000 RWF. Fee was 0 RWF.Kanda*182*16# wiyandikishe muri poromosiyo ya BivaMoMotima, ugire amahirwe yo gutsindira i
bihembo bishimishije."}, {"transaction_id": 3, "original_transaction_id": "51732411227", "category": "Payment", "amount": 600
, "fee": 0, "balance_after": 400, "transaction_date": "2024-05-10 21:32:32", "status": "Success", "sender": "Abebe Chala CHEB
UDIE", "receiver": "Samuel Carter", "raw_sms": "TxId: 51732411227. Your payment of 600 RWF to Samuel Carter 95464 has been co
mpleted at 2024-05-10 21:32:32. Your new balance: 400 RWF. Fee was 0 RWF.Kanda*182*16# wiyandikishe muri poromosiyo ya BivaMo
Motima, ugire amahirwe yo gutsindira ibihembo bishimishije."}, {"transaction_id": 4, "original_transaction_id": null, "catego
ry": "Bank Deposit", "amount": 40000, "fee": null, "balance_after": 40400, "transaction_date": "2024-05-11 18:43:49", "status
": "Success", "sender": null, "receiver": "Abebe Chala CHEBUDIE", "raw_sms": "*113*R*A bank deposit of 40000 RWF has been add
ed to your mobile money account at 2024-05-11 18:43:49. Your NEW BALANCE :40400 RWF. Cash Deposit::CASH:::0:250795963036.Th
a00 65280 0 65280 0 0 226k 0 --:--:-- --:--:-- --:--:-- 227k
curl: (23) Failure writing output to destination, passed 16384 returned 256
```

GET /transactions/{id}

Returns a single transaction by its auto-generated ID.

Request: curl -u admin:MomoSms26 <http://localhost:8080/transactions/1>

```
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$ curl -u admin:MomoSms26 http://localhost:8080/transactions/1
{"transaction_id": 1, "original_transaction_id": "76662021700", "category": "Incoming Money", "amount": 2000, "fee": null, "b
alance_after": 2000, "transaction_date": "2024-05-10 16:30:51", "status": "Success", "sender": "Jane Smith", "receiver": "Abe
be Chala CHEBUDIE", "raw_sms": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 202
4-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$
```

POST /transactions

Adds a new transaction. Required fields are transaction_id, category, amount, sender, receiver and transaction_date.

Request:

```
curl -u admin:MomoSms26 -X POST -H "Content-Type: application/json" -d
'{"transaction_id":9999,"category":"Payment","amount":5000,"sender":"Account
Holder","receiver":"Samuel Carter","transaction_date":"2024-06-02 10:00:00"}'
http://localhost:8080/transactions
```

```

Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$ curl -u admin:MomoSms26 -X POST -H "Content-Type: application/json" -d '{"transaction_id":9999,"category":"Payment",\
"amount":5000,"sender":"Account Holder","receiver":"Samuel Carter","transaction_date":"2024-06-02 10:00:00","fee\
e":0,"balance_after":1000,"status":"Success","raw_sms":"Your payment of 5000 RWF to Samuel Carter has been complete\
d"}' http://localhost:8080/transactions
{"transaction_id": 9999, "category": "Payment", "amount": 5000, "sender": "Account Holder", "receiver": "Samuel Carter", "tra\
nsaction_date": "2024-06-02 10:00:00", "fee": 0, "balance_after": 1000, "status": "Success", "raw_sms": "Your payment of 5000\
RWF to Samuel Carter has been completed"}
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$

```

PUT /transactions/{id}

Updates specific fields of an existing transaction. Only include the fields you want to change.

Request: curl -u admin:MomoSms26 -X PUT -H "Content-Type: application/json" -d '{"amount":9999}' <http://localhost:8080/transactions/1>

```

Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$ curl -u admin:MomoSms26 -X PUT -H "Content-Type: application/json" -d '{"amount":9999}' http://localhost:8080/transaction\
s/1
{"transaction_id": 1, "original_transaction_id": "76662021700", "category": "Incoming Money", "amount": 9999, "fee": null, "b\
alance_after": 2000, "transaction_date": "2024-05-10 16:30:51", "status": "Success", "sender": "Jane Smith", "receiver": "Abe\
be Chala CHEBUDIE", "raw_sms": "You have received 2000 RWF from Jane Smith (*****013) on your mobile money account at 202\
4-05-10 16:30:51. Message from sender: . Your new balance:2000 RWF. Financial Transaction Id: 76662021700."}
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$

```

DELETE /transactions/{id}

Permanently removes a transaction from the system.

Request: curl -u admin:MomoSms26 -X DELETE <http://localhost:8080/transactions/9999>

```

Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$ curl -u admin:MomoSms26 -X DELETE http://localhost:8080/transactions/9999
{"message": "Transaction deleted successfully"}
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$

```

DSA Comparison Results

To efficiently retrieve transactions by ID two search algorithms were implemented and compared against 1,660 real MoMo transaction records parsed from modified_sms_v2.xml.

Algorithm 1 — Linear Search $O(n)$

Linear Search iterates through the transactions list one record at a time until it finds the matching transaction_id. In the worst case where the target is at the end or does not exist it checks every single record. With 1,660 transactions this means up to 1,660 comparisons per search.

```
'''
```

```
python
```

```
def linear_search(transactions, target_id):  
    for transaction in transactions:  
        if transaction["transaction_id"] == target_id:  
            return transaction  
    return None
```

```
''
```

Algorithm 2 — Dictionary Lookup $O(1)$

Dictionary Lookup stores all transactions in a Python dictionary with transaction_id as the key. Python dictionaries use hash tables which calculate the exact memory location of a value directly from its key. This means the lookup time is constant regardless of how many transactions exist.

```
'''python
```

```
def dictionary_lookup(transactions_dict, target_id):  
    return transactions_dict.get(target_id, None)
```

```
'''
```

Performance Results

Each algorithm was tested on 20 target IDs covering early, middle, late and non-existent IDs. Each test was repeated 10,000 times to produce accurate measurable timing results.

| Transaction ID | Position | Linear Search | Dictionary Lookup | Speedup |

```
Ange@DESKTOP-E9VD2VE MINGW64 ~/OneDrive/Documentos/Momo-SMS-Data-Visualization (main)
$ python dsa/search_comparison.py
Total transactions loaded: 1660
```

```
Searching for 20 transaction IDs...
```

ID	Linear Search	Dictionary Lookup	Speed Difference
1	0.003028s (Found)	0.002461s	1.2x faster
5	0.005307s (Found)	0.001837s	2.9x faster
10	0.009906s (Found)	0.002584s	3.8x faster
25	0.014563s (Found)	0.001488s	9.8x faster
50	0.023383s (Found)	0.001245s	18.8x faster
100	0.044191s (Found)	0.001250s	35.4x faster
200	0.086276s (Found)	0.001247s	69.2x faster
300	0.131335s (Found)	0.001204s	109.1x faster
400	0.180512s (Found)	0.001531s	117.9x faster
500	0.229528s (Found)	0.001252s	183.3x faster
600	0.297491s (Found)	0.001521s	195.6x faster
800	0.351261s (Found)	0.001538s	228.3x faster
1000	0.443901s (Found)	0.001356s	327.4x faster
1200	0.542649s (Found)	0.001478s	367.0x faster
1400	0.638482s (Found)	0.001207s	528.9x faster
1500	0.676029s (Found)	0.003502s	193.0x faster
1600	0.717362s (Found)	0.001210s	593.1x faster
1650	0.754733s (Found)	0.001194s	632.4x faster
1661	0.765628s (Not Found)	0.001433s	534.3x faster
9999	0.715533s (Not Found)	0.001179s	607.1x faster

=====
Total Linear Search Time: 6.631098s
Total Dictionary Lookup Time: 0.031717s
Overall Speedup: 209.1x faster

Dictionary lookup was consistently faster across all test cases. The advantage grows significantly as the target ID increases because linear search must scan further while dictionary lookup always jumps directly to the result.

Why Dictionary Lookup Is Faster

Linear Search time grows proportionally with the number of records — $O(n)$. If the number of transactions doubles the search time doubles.

Dictionary Lookup time stays constant regardless of list size $O(1)$. Whether there are 100 or 1,000,000 transactions the lookup takes the same amount of time because Python's hash function calculates the exact memory location of the value directly from the key.

The tradeoff is that building the dictionary requires $O(n)$ time and extra memory upfront. But once built every subsequent lookup is instant.

Alternative Data Structure

A Binary Search Tree (BST) or balanced variant such as a Red-Black Tree could also improve search efficiency. Transactions stored in a BST sorted by `transaction_id` would take $O(\log n)$ per search. For 1,660 records this means checking at most 11 items instead of all 1,660. While slower than dictionary lookup a BST maintains sorted order which makes it useful for range queries such as finding all transactions between ID 500 and ID 800.

Basic Authentication Limitations and Alternatives

What Basic Authentication Is

Basic Authentication is the simplest form of HTTP authentication. The client combines the username and password separated by a colon and encodes the result in base64. This encoded string is sent with every request in the Authorization header.

For example `admin:MomoSms26` becomes `YWRtaW46TW9tb1NtczI2` in base64 and is sent as:

Why Basic Authentication Is Weak

Base64 is not encryption. Anyone can decode a base64 string instantly. If an attacker intercepts a request they immediately recover the original username and password.

Credentials sent with every request. Unlike token-based systems Basic Auth sends the actual password with every single request increasing the risk of interception.

No expiry mechanism. Credentials never expire automatically. A leaked password gives an attacker permanent access until manually changed.

No token revocation. There is no way to invalidate a compromised session without changing the password for all users.

Vulnerable to brute force. Without rate limiting attackers can try thousands of credential combinations automatically.

Stronger Alternatives

JWT — JSON Web Tokens

After a successful login the server generates a signed token containing the user identity and an expiry time. The client stores this token and sends it with subsequent requests. The actual password is never sent again after the initial login. Tokens expire automatically and can be revoked without requiring a password change.

OAuth2

OAuth2 is an industry standard authorization framework used by Google, GitHub and Facebook. Users authorize applications to access their data through short-lived access tokens issued

by an authorization server. The actual password is never shared with the application. Access can be granted for specific scopes only and revoked at any time.

Recommendation

For a production MoMo SMS API handling real financial transaction data Basic Authentication is not sufficient. The recommended approach is JWT authentication combined with HTTPS encryption. This ensures credentials are never exposed in transit, sessions expire automatically, and access can be revoked without disrupting other users.

Conclusion

This assignment demonstrated how to build and secure a REST API using plain Python without any frameworks. The MoMo SMS API successfully implements all five CRUD endpoints protected by Basic Authentication. The DSA comparison confirmed that dictionary lookup with $O(1)$ complexity is significantly more efficient than linear search with $O(n)$ complexity for retrieving transactions by ID. The reflection on Basic Auth limitations highlights the importance of adopting stronger authentication mechanisms such as JWT or OAuth2 in production financial systems where data security is critical.